

```
"""
```

```
LinkedIn Job Bot – Pratik Wadaskar
```

```
Features:
```

- Multi-keyword search (Backend Engineer, SDE, Node.js Developer)
- NLP-based resume-JD matching (updated resume)
- Easy Apply automation with form filling
- External ATS handlers: Greenhouse, Workday, Lever, iCIMS
- Unknown portals: saved to manual_apply.csv
- All results logged to applied_jobs.csv

```
"""
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from sentence_transformers import SentenceTransformer, util
from keybert import KeyBERT
import spacy
import time
import csv
import pickle
import os
import re
import sys

# -----
# CONFIG – edit these
# -----

LINKEDIN_EMAIL    = "pratikwadaskar123@gmail.com"
LINKEDIN_PASSWORD = "#Theboys@123"
RESUME_PATH       = os.path.join(os.path.dirname(__file__), "Pratik_Wadaskar_Resu
COOKIES_FILE      = "linkedin_cookies.pkl"
MATCH_THRESHOLD   = 0.55          # jobs below this score are skipped
EXPERIENCE_YEARS  = "3"           # used in text inputs
SALARY_EXPECTED   = "1700000"    # ~17 LPA in INR (numeric, used in salary fields)
NOTICE_PERIOD     = "30"         # days
LOCATION_PREF      = "Open to any location"

PHONE    = "9049731756"
CITY     = "Pune"
STATE    = "Maharashtra"
COUNTRY  = "India"
```

```
# Job titles to search (runs a separate pass for each)
JOB_TITLES = [
    "Backend Engineer",
    "Software Development Engineer",
    "Node.js Developer",
]

# -----
# RESUME TEXT (updated to current resume)
# -----
RESUME_TEXT = """
Pratik Wadaskar
Pune | pratikwadaskar9049@gmail.com | +91 9049731756 | linkedin.com/in/pratik-wad
```

Summary

Backend Engineer with 3+ years of experience building production-grade microservices. Hands-on with ISO 20022 payment message standards, RabbitMQ-driven priority queue, database optimization. Experienced delivering features in regulated, compliance-sensitive, security, and system stability are non-negotiable.

Skills

Language & Backend: Node.js, Express.js, Python, REST APIs, Mocha, Chai, Sinon
Messaging & Architecture: RabbitMQ, Microservices, Event-Driven Systems, Distributed
Frontend: React.js, JavaScript (ES6+), Redux
Databases: Oracle SQL (partitioning, indexing), PostgreSQL, Redis
DevOps & Tools: Docker, Jenkins CI/CD, Git, GitHub, PM2, SonarQube, AWS (S3, EC2)

Experience

Product Developer, EdgeVerve Systems (Finacle) – Pune Aug 2025 – Present

- Implemented Business Priority (P1/P2/P3) message routing in a RabbitMQ-driven pipeline, moving the logic into a shared Node.js module consumed across payment microservices.
- Resolved a defect in the ISO 20022 pacs.004 payment message mapper – identified and corrected AutobaML mapping to align with international payment message standards.
- Eliminated a Severity-1 data privacy compliance violation by removing PII from logs.
- Built a Redis cache sync mechanism for the Holiday Calendar model exposing on-demand data.
- Improved SonarQube code coverage from ~45% to ~80% across payment microservices.

Software Development Engineer, Jio Platforms Limited – Navi Mumbai Dec 2022 – Aug 2025

- Architected a full-stack CDR processing pipeline for internet data volume recording, integrating Oracle DB tables supporting inbound/outbound roaming and home outbound reconciliation.
- Optimized CDR file ingestion reducing processing time from ~3 minutes to ~25 seconds.
- Authored a data validation module with regex-based filename verification and AEI checks.
- Built Python aggregation scripts for UDR, USR, and BSR billing reports shared with finance.
- Implemented date-based Oracle partitioning, query indexing, and server-side pagination.

Education

Projects

AI-Based Job Matching Automation Tool - Python, NLP

Real-Time Speed Monitoring System - Dockerized Microservices, PostgreSQL, WebSock
"""

```
# -----  
# NLP SETUP  
# -----  
  
print("🕒 Loading NLP models...")  
model = SentenceTransformer("all-MiniLM-L6-v2")  
kw_model = KeyBERT(model)  
nlp = spacy.load("en_core_web_sm")  
  
CANONICAL_SKILLS = [  
    "node.js", "express.js", "python", "javascript", "typescript",  
    "rest api", "microservices", "rabbitmq", "event-driven", "distributed systems",  
    "redis", "oracle sql", "postgresql", "mongodb", "mysql",  
    "docker", "kubernetes", "jenkins", "ci/cd", "aws", "s3", "ec2",  
    "react.js", "redux", "html", "css",  
    "git", "github", "postman", "sonarqube", "pm2",  
    "mocha", "chai", "sinon", "unit testing",  
    "iso 20022", "fintech", "payments", "banking", "compliance", "gdpr",  
    "kafka", "grpc", "graphql", "spring boot", "java",  
    "backend", "backend developer", "backend engineer", "software engineer",  
    "api development", "database optimization", "performance optimization",  
    "agile", "scrum", "system design", "scalability",  
]  
  
canonical_embeddings = model.encode(CANONICAL_SKILLS, convert_to_tensor=True)  
  
def clean_text(text):  
    text = re.sub(r"^[a-zA-Z0-9\s.,]", " ", text)  
    return re.sub(r"\s+", " ", text).strip().lower()  
  
def normalize_keyword(kw, threshold=0.62):  
    kw_vec = model.encode(kw, convert_to_tensor=True)  
    sims = util.cos_sim(kw_vec, canonical_embeddings)[0]  
    idx = int(sims.argmax())  
    if sims[idx].item() >= threshold:  
        return CANONICAL_SKILLS[idx]  
    return None  
  
def extract_structured_keywords(text):
```

```

doc = nlp(text)
candidates = set()
for chunk in doc.noun_chunks:
    phrase = chunk.text.lower().strip()
    if 1 <= len(phrase) <= 40:
        candidates.add(phrase)
for token in doc:
    if token.pos_ in {"PROPN", "NOUN"} and not token.is_stop:
        candidates.add(token.text.lower())
normalized = set()
for kw in candidates:
    norm = normalize_keyword(kw)
    if norm:
        normalized.add(norm)
return normalized

def fuzzy_match_keywords(set1, set2, threshold=0.7):
    matched = set()
    for kw1 in set1:
        for kw2 in set2:
            sim = util.cos_sim(model.encode(kw1), model.encode(kw2)).item()
            if sim > threshold:
                matched.add(kw1)
                break
    return matched

def chunk_text(text, max_words=150):
    words = text.split()
    return [" ".join(words[i:i+max_words]) for i in range(0, len(words), max_word

# Pre-compute resume embeddings once
resume_keywords = extract_structured_keywords(RESUME_TEXT)
resume_chunks = [clean_text(c) for c in chunk_text(RESUME_TEXT)]
resume_chunk_embs = model.encode(resume_chunks, convert_to_tensor=True)
print(f"✅ Resume loaded - {len(resume_keywords)} canonical keywords extracted")

# -----
# SCORE A JOB
# -----

def score_job(jd_text):
    clean_jd = clean_text(jd_text)
    jd_keywords = extract_structured_keywords(clean_jd)
    matched = fuzzy_match_keywords(resume_keywords, jd_keywords)

```

```

tech_score = len(matched) / len(jd_keywords) if jd_keywords else 0

jd_chunks      = [clean_text(c) for c in chunk_text(clean_jd)]
jd_chunk_embs  = model.encode(jd_chunks, convert_to_tensor=True)
sims           = util.pytorch_cos_sim(resume_chunk_embs, jd_chunk_embs)[0]
max_sim        = sims.max().item()

final = (0.4 * max_sim) + (0.6 * tech_score)
return round(final, 3), round(max_sim, 3), round(tech_score, 3), sorted(match

# -----
#  BROWSER SETUP
# -----

def make_driver():
    opts = Options()
    opts.add_argument("--start-maximized")
    opts.add_argument("--disable-blink-features=AutomationControlled")
    opts.add_experimental_option("excludeSwitches", ["enable-logging", "enable-au
    opts.add_experimental_option("useAutomationExtension", False)
    opts.add_argument("--no-sandbox")
    opts.add_argument("--disable-dev-shm-usage")
    return webdriver.Chrome(options=opts)

# -----
#  LINKEDIN LOGIN
# -----

def linkedin_login(driver, wait):
    driver.get("https://www.linkedin.com")

    if os.path.exists(COOKIES_FILE):
        try:
            cookies = pickle.load(open(COOKIES_FILE, "rb"))
            for cookie in cookies:
                driver.add_cookie(cookie)
            driver.get("https://www.linkedin.com/feed")
            wait.until(EC.url_contains("/feed"))
            print("✅ Logged in via saved cookies")
            return
        except Exception as e:
            print(f"⚠️ Cookie login failed ({e}), falling back to password login

    driver.get("https://www.linkedin.com/login")
    wait.until(EC.presence_of_element_located((By.ID, "username"))).send_keys(LIN
    driver.find_element(By.ID, "password").send_keys(LINKEDIN_PASSWORD + Keys.RET
    wait.until(EC.url_contains("/feed"))

```

```

pickle.dump(driver.get_cookies(), open(COOKIES_FILE, "wb"))
print("✅ Logged in and cookies saved")
time.sleep(3)

# -----
# SCRAPE JOB URLS (multi-page)
# -----
def scrape_job_urls(driver, wait, job_title):
    search_url = (
        "https://www.linkedin.com/jobs/search/?"
        f"keywords={job_title.replace(' ', '%20')}}"
        "&location=India"
        "&f_TP=1"
        "&f_TPR=r86400"
        "&f_E=2%2C3"          # Entry / Associate
        "&f_JT=F"             # Full-time
        "&sortBy=DD"
    )
    driver.get(search_url)
    time.sleep(4)

    job_urls = set()
    page_num = 1

    while True:
        print(f"📄 Page {page_num} - '{job_title}'")
        try:
            scaffold = wait.until(EC.presence_of_element_located((By.CLASS_NAME,
                                                                    scrollable = scaffold.find_element(By.XPATH, "./div")
            wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, ".jo
        except Exception:
            break

        last_h = 0
        for _ in range(120):
            driver.execute_script("arguments[0].scrollTop += 300;", scrollable)
            time.sleep(0.35)
            cards = scrollable.find_elements(By.CSS_SELECTOR, ".job-card-containe
            for card in cards:
                href = card.get_attribute("href")
                if href and "/jobs/view/" in href:
                    job_urls.add(href.split("?")[0])
            cur_h = driver.execute_script("return arguments[0].scrollTop", scroll
            if cur_h == last_h:
                break
            last_h = cur_h

```

```

        # Next page
        try:
            nxt = driver.find_element(By.CSS_SELECTOR, "button.jobs-search-pagina
            if not nxt.is_enabled():
                break
            driver.execute_script("arguments[0].click();", nxt)
            time.sleep(3)
            page_num += 1
        except Exception:
            break

    print(f" 🔍 {len(job_urls)} URLs collected for '{job_title}'")
    return job_urls

# -----
# EASY APPLY HANDLER
# -----
def safe_fill(driver, wait, css, value, field_type="input"):
    """Try to fill a field; silently skip if not found."""
    try:
        el = WebDriverWait(driver, 4).until(EC.presence_of_element_located((By.CSS_SELECTOR, css)))
        el.clear()
        el.send_keys(value)
        return True
    except Exception:
        return False

def fill_easy_apply_form(driver, wait):
    """
    Iterates through multi-step Easy Apply modal.
    Fills known fields, handles radio buttons, dropdowns, file upload.
    Returns 'submitted', 'skipped', or 'error'.
    """
    max_steps = 10
    for step in range(max_steps):
        time.sleep(1.5)

        # — Phone number —
        for ph_css in [
            "input[id*='phoneNumber']",
            "input[name*='phone']",
            "input[placeholder*='Phone']",
        ]:
            safe_fill(driver, wait, ph_css, PHONE)

```

```

# — Resume upload —
try:
    upload_input = driver.find_element(By.CSS_SELECTOR, "input[type='file'
    if os.path.exists(RESUME_PATH):
        upload_input.send_keys(os.path.abspath(RESUME_PATH))
        time.sleep(2)
except Exception:
    pass

# — Text fields - experience, notice, salary —
text_inputs = driver.find_elements(By.CSS_SELECTOR, "input[type='text']",
for inp in text_inputs:
    try:
        label_text = ""
        # Try to find associated label
        try:
            inp_id = inp.get_attribute("id")
            if inp_id:
                lbl = driver.find_element(By.CSS_SELECTOR, f"label[for='{
                label_text = lbl.text.lower()
        except Exception:
            pass

        aria = (inp.get_attribute("aria-label") or "").lower()
        placeholder = (inp.get_attribute("placeholder") or "").lower()
        combined = label_text + " " + aria + " " + placeholder

        if any(k in combined for k in ["year", "experience", "exp"]):
            if not inp.get_attribute("value"):
                inp.clear(); inp.send_keys(EXPERIENCE_YEARS)
        elif any(k in combined for k in ["salary", "ctc", "expected", "lp
            if not inp.get_attribute("value"):
                inp.clear(); inp.send_keys(SALARY_EXPECTED)
        elif any(k in combined for k in ["notice", "joining"]):
            if not inp.get_attribute("value"):
                inp.clear(); inp.send_keys(NOTICE_PERIOD)
        elif any(k in combined for k in ["city", "location"]):
            if not inp.get_attribute("value"):
                inp.clear(); inp.send_keys(CITY)
        elif any(k in combined for k in ["linkedin", "profile"]):
            val = inp.get_attribute("value") or ""
            if not val:
                inp.clear(); inp.send_keys("https://linkedin.com/in/prati
        elif any(k in combined for k in ["github", "portfolio"]):
            val = inp.get_attribute("value") or ""
            if not val:
                inp.clear(); inp.send_keys("https://github.com/pratikw123

```



```

except Exception:
    continue

# — Dropdowns —
selects = driver.find_elements(By.TAG_NAME, "select")
for sel_el in selects:
    try:
        sel = Select(sel_el)
        label_text = ""
        try:
            sel_id = sel_el.get_attribute("id")
            if sel_id:
                lbl = driver.find_element(By.CSS_SELECTOR, f"label[for='{
                    label_text = lbl.text.lower()
        except Exception:
            pass
        aria = (sel_el.get_attribute("aria-label") or "").lower()
        combined = label_text + " " + aria

        if any(k in combined for k in ["year", "experience"]):
            # Pick option closest to 2-4 years
            options = [o.text for o in sel.options]
            for pref in ["3", "2-4", "3-5", "2", "4", "1-3"]:
                match = next((o for o in options if pref in o), None)
                if match:
                    sel.select_by_visible_text(match)
                    break
        elif any(k in combined for k in ["notice", "joining"]):
            options = [o.text for o in sel.options]
            for pref in ["30", "1 month", "30 days", "one month"]:
                match = next((o for o in options if pref.lower() in o.lower()), None)
                if match:
                    sel.select_by_visible_text(match)
                    break
        elif any(k in combined for k in ["country"]):
            try:
                sel.select_by_visible_text("India")
            except Exception:
                pass
        elif any(k in combined for k in ["relocat"]):
            try:
                sel.select_by_visible_text("Yes")
            except Exception:
                pass
    except Exception:
        continue

```

```

# — Yes/No radio — relocate, authorized, etc —
try:
    radios = driver.find_elements(By.CSS_SELECTOR, "input[type='radio']")
    for radio in radios:
        val = (radio.get_attribute("value") or "").lower()
        aria = (radio.get_attribute("aria-label") or "").lower()
        if "yes" in val or "yes" in aria or "true" in val:
            if not radio.is_selected():
                driver.execute_script("arguments[0].click();", radio)
except Exception:
    pass

# — Find the action button —
# Priority: Submit > Review > Next > Continue
submitted = False
for btn_text in ["Submit application", "Submit", "Review", "Next", "Conti
try:
    btn = driver.find_element(
        By.XPATH,
        f"//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    )
    if btn.is_displayed() and btn.is_enabled():
        driver.execute_script("arguments[0].click();", btn)
        time.sleep(1.5)
        if "submit" in btn_text.lower():
            submitted = True
            break
except Exception:
    continue

if submitted:
    # Dismiss confirmation modal if shown
    time.sleep(2)
    try:
        dismiss = driver.find_element(By.CSS_SELECTOR, "button[aria-label
        dismiss.click()
    except Exception:
        pass
    return "submitted"

# Check if modal closed (application done)
try:
    driver.find_element(By.CSS_SELECTOR, "div.jobs-easy-apply-modal")
except Exception:
    return "submitted"

return "skipped"

```

```

def try_easy_apply(driver, wait):
    """Click Easy Apply button and run the form filler."""
    try:
        btn = wait.until(EC.element_to_be_clickable(
            (By.CSS_SELECTOR, "button.jobs-apply-button[aria-label*='Easy Apply']")
        ))
        driver.execute_script("arguments[0].click();", btn)
        time.sleep(2)
        return fill_easy_apply_form(driver, wait)
    except Exception as e:
        return f"error: {e}"

```

```

# -----
#  EXTERNAL ATS HANDLERS
#  -----

```

```

PROFILE_DATA = {
    "first_name":    "Pratik",
    "last_name":     "Wadaskar",
    "full_name":     "Pratik Wadaskar",
    "email":         "pratikwadaskar9049@gmail.com",
    "phone":         PHONE,
    "city":          CITY,
    "state":         STATE,
    "country":       COUNTRY,
    "linkedin":      "https://linkedin.com/in/pratik-wadaskar",
    "github":        "https://github.com/pratikw123",
    "years_exp":     EXPERIENCE_YEARS,
    "salary":        SALARY_EXPECTED,
    "notice_period": NOTICE_PERIOD,
}

```

```

def fill_generic_fields(driver):
    """Best-effort fill for any ATS – maps common field patterns to profile data.
    field_map = {
        # name
        ("first", "fname", "firstname"):    PROFILE_DATA["first_name"],
        ("last", "lname", "lastname", "surname"): PROFILE_DATA["last_name"],
        ("full_name", "fullname", "name"):   PROFILE_DATA["full_name"],
        # contact
        ("email",): PROFILE_DATA["email"],
        ("phone", "mobile", "telephone"):    PROFILE_DATA["phone"],
        # location
    }

```

```

("city",): PROFILE_DATA["city"],
("state",): PROFILE_DATA["state"],
("country",): PROFILE_DATA["country"]
# links
("linkedin",): PROFILE_DATA["linkedin"]
("github", "portfolio", "website"): PROFILE_DATA["github"],
# experience / salary
("year", "experience", "exp"): PROFILE_DATA["years_exp"]
("salary", "ctc", "compensation", "expected"): PROFILE_DATA["salary"],
("notice", "joining"): PROFILE_DATA["notice_pe"]
}

```

```

inputs = driver.find_elements(By.CSS_SELECTOR, "input[type='text']", input[typ
for inp in inputs:

```

```

    try:
        if not inp.is_displayed():
            continue
        label_text = ""
        try:
            inp_id = inp.get_attribute("id") or ""
            if inp_id:
                lbl = driver.find_element(By.CSS_SELECTOR, f"label[for='{inp_
                label_text = lbl.text.lower()
        except Exception:
            pass
        aria = (inp.get_attribute("aria-label") or "").lower()
        ph = (inp.get_attribute("placeholder") or "").lower()
        name = (inp.get_attribute("name") or "").lower()
        combined = f"{label_text} {aria} {ph} {name}"

        for keywords, value in field_map.items():
            if any(k in combined for k in keywords):
                current = inp.get_attribute("value") or ""
                if not current.strip():
                    inp.clear()
                    inp.send_keys(value)
                break
    except Exception:
        continue

```

```

# Resume upload
try:
    upload = driver.find_element(By.CSS_SELECTOR, "input[type='file']")
    if os.path.exists(RESUME_PATH):
        upload.send_keys(os.path.abspath(RESUME_PATH))
        time.sleep(2)
except Exception:

```

pass

```
def detect_ats(url):
    url = url.lower()
    if "greenhouse.io" in url or "boards.greenhouse" in url:
        return "greenhouse"
    if "myworkdayjobs.com" in url or "workday.com" in url:
        return "workday"
    if "lever.co" in url:
        return "lever"
    if "icims.com" in url:
        return "icims"
    if "smartrecruiters.com" in url:
        return "smartrecruiters"
    if "jobs.google.com" in url or "careers.google.com" in url:
        return "google"
    if "amazon.jobs" in url:
        return "amazon"
    if "careers.microsoft.com" in url:
        return "microsoft"
    return "unknown"

def apply_greenhouse(driver, wait):
    try:
        fill_generic_fields(driver)
        time.sleep(1)
        # Greenhouse submit button
        btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "input[type
        driver.execute_script("arguments[0].click();", btn)
        time.sleep(2)
        return "submitted"
    except Exception as e:
        return f"error: {e}"

def apply_lever(driver, wait):
    try:
        fill_generic_fields(driver)
        time.sleep(1)
        btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[typ
        driver.execute_script("arguments[0].click();", btn)
        time.sleep(2)
        return "submitted"
    except Exception as e:
        return f"error: {e}"
```

```

def apply_workday(driver, wait):
    """Workday is complex/dynamic – fill what we can, flag for review."""
    try:
        fill_generic_fields(driver)
        time.sleep(2)
        return "partial" # Workday usually needs CAPTCHA / manual review
    except Exception as e:
        return f"error: {e}"

def apply_external(driver, wait, apply_url):
    """Open external apply URL and attempt to fill based on detected ATS."""
    driver.execute_script(f"window.open('{apply_url}', '_blank');")
    time.sleep(3)
    driver.switch_to.window(driver.window_handles[-1])
    time.sleep(3)

    current_url = driver.current_url
    ats = detect_ats(current_url)
    print(f"🔧 Detected ATS: {ats} – {current_url[:80]}")

    result = "unknown_ats"
    if ats == "greenhouse":
        result = apply_greenhouse(driver, wait)
    elif ats == "lever":
        result = apply_lever(driver, wait)
    elif ats == "workday":
        result = apply_workday(driver, wait)
    elif ats in ("google", "amazon", "microsoft", "icims", "smartrecruiters", "un
        # Best-effort generic fill
        fill_generic_fields(driver)
        time.sleep(1)
        result = "manual_needed"

    # Close the tab, go back to LinkedIn tab
    driver.close()
    driver.switch_to.window(driver.window_handles[0])
    return result, current_url

# -----
# GET JD TEXT + APPLY BUTTON TYPE
# -----

def get_job_details(driver, wait, job_url):
    driver.get(job_url)

```

```

time.sleep(3)

jd_text = ""
apply_type = "unknown" # "easy_apply" | "external" | "unknown"
apply_url = ""
job_title = ""
company = ""

try:
    job_title = driver.find_element(By.CSS_SELECTOR, "h1.t-24").text
except Exception:
    pass

try:
    company = driver.find_element(By.CSS_SELECTOR, "a.app-aware-link span[dir
except Exception:
    pass

# JD text
try:
    jd_el = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "arti
    # Click "See more"
    try:
        see_more = WebDriverWait(driver, 3).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(@class, '
        )
        driver.execute_script("arguments[0].click();", see_more)
        time.sleep(1)
    except Exception:
        pass
    jd_text = jd_el.text
except Exception:
    pass

# Detect apply type
try:
    driver.find_element(By.CSS_SELECTOR, "button.jobs-apply-button[aria-label
    apply_type = "easy_apply"
except Exception:
    try:
        ext_btn = driver.find_element(By.CSS_SELECTOR, "button.jobs-apply-but
        apply_type = "external"
        # Get actual external URL by clicking and capturing redirect
        driver.execute_script("arguments[0].click();", ext_btn)
        time.sleep(2)
        # Look for redirect in new tab or current page
        if len(driver.window_handles) > 1:
            driver.switch_to.window(driver.window_handles[-1])

```

```

        apply_url = driver.current_url
        driver.close()
        driver.switch_to.window(driver.window_handles[0])
    else:
        apply_url = driver.current_url
        driver.back()
        time.sleep(2)
    except Exception:
        apply_type = "unknown"

    return jd_text, apply_type, apply_url, job_title, company

# -----
# LOGGING
# -----
APPLIED_LOG    = "applied_jobs.csv"
MANUAL_LOG     = "manual_apply.csv"
SKIPPED_LOG    = "skipped_jobs.csv"

def init_logs():
    for filepath, headers in [
        (APPLIED_LOG, ["job_url", "job_title", "company", "apply_type", "ats", "s
        (MANUAL_LOG,  ["job_url", "job_title", "company", "apply_url", "score"]),
        (SKIPPED_LOG, ["job_url", "job_title", "company", "score", "reason"]),
    ]:
        if not os.path.exists(filepath):
            with open(filepath, "w", newline="", encoding="utf-8") as f:
                csv.writer(f).writerow(headers)

def log_applied(job_url, job_title, company, apply_type, ats, score, status):
    with open(APPLIED_LOG, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([job_url, job_title, company, apply_type, ats, sco

def log_manual(job_url, job_title, company, apply_url, score):
    with open(MANUAL_LOG, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([job_url, job_title, company, apply_url, score])

def log_skipped(job_url, job_title, company, score, reason):
    with open(SKIPPED_LOG, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([job_url, job_title, company, score, reason])

# -----
# MAIN
# -----
def main():

```



```

init_logs()
driver = make_driver()
wait = WebDriverWait(driver, 20)

linkedin_login(driver, wait)

# Collect all job URLs across all search titles
all_job_urls = set()
for title in JOB_TITLES:
    urls = scrape_job_urls(driver, wait, title)
    all_job_urls.update(urls)

print(f"\n🎯 Total unique job URLs: {len(all_job_urls)}\n")

applied_count = 0
manual_count = 0
skipped_count = 0
already_applied = set()

# Load previously applied URLs to avoid duplicates
if os.path.exists(APPLIED_LOG):
    with open(APPLIED_LOG, newline="", encoding="utf-8") as f:
        for row in csv.reader(f):
            if row:
                already_applied.add(row[0])

for i, job_url in enumerate(all_job_urls):
    if job_url in already_applied:
        print(f"[{i+1}] 🔄 Already applied - skip")
        continue

    print(f"\n[{i+1}/{len(all_job_urls)}] 🔍 {job_url}")

    try:
        jd_text, apply_type, apply_url, job_title, company = get_job_details(
    except Exception as e:
        print(f"⚠️ Could not load job page: {e}")
        continue

    if not jd_text:
        print(f"⚠️ Empty JD - skipping")
        log_skipped(job_url, job_title, company, 0, "empty_jd")
        skipped_count += 1
        continue

    final_score, sem_score, tech_score, matched_kws = score_job(jd_text)
    print(f"🇩🇪 Score: {final_score} (semantic: {sem_score}, tech: {tech_sc

```

```

print(f" 🟡 Matched: {matched_kws[:8]}")

if final_score < MATCH_THRESHOLD:
    print(f" ❌ Below threshold ({MATCH_THRESHOLD}) - skipping")
    log_skipped(job_url, job_title, company, final_score, "low_score")
    skipped_count += 1
    continue

print(f" ✅ '{job_title}' @ '{company}' - apply_type: {apply_type}")

# — Easy Apply —
if apply_type == "easy_apply":
    status = try_easy_apply(driver, wait)
    print(f" 🟡 Easy Apply result: {status}")
    log_applied(job_url, job_title, company, "easy_apply", "linkedin", fi
    applied_count += 1

# — External Apply —
elif apply_type == "external" and apply_url:
    ats = detect_ats(apply_url)
    if ats in ("google", "amazon", "microsoft") or ats == "unknown":
        # Save for manual
        print(f" 📁 Saved to manual_apply.csv (ATS: {ats})")
        log_manual(job_url, job_title, company, apply_url, final_score)
        log_applied(job_url, job_title, company, "external", ats, final_s
        manual_count += 1
    else:
        result, resolved_url = apply_external(driver, wait, apply_url)
        print(f" 🟡 External result: {result}")
        log_applied(job_url, job_title, company, "external", ats, final_s
        if result == "manual_needed":
            log_manual(job_url, job_title, company, resolved_url, final_s
            manual_count += 1
        else:
            applied_count += 1

else:
    print(f" ⚠️ Could not determine apply path - saving to manual")
    log_manual(job_url, job_title, company, job_url, final_score)
    manual_count += 1

time.sleep(2) # Polite delay between applications

driver.quit()

print("\n" + "="*50)
print(f" ✅ Done!")

```

```
print(f"    Applied (automated): {applied_count}")
print(f"    Saved for manual:    {manual_count}")
print(f"    Skipped (low score): {skipped_count}")
print(f"\n📄 Logs:")
print(f"    {APPLIED_LOG}")
print(f"    {MANUAL_LOG}")
print(f"    {SKIPPED_LOG}")
```

```
if __name__ == "__main__":
    main()
```